Multi-class SVM & Support Vector Regression (SVR)

Contents

10.1 Multi-class SVM **1**
10.1.1 One-vs-rest (one-vs-all) . . . 2
10.1.2 One-vs-one (all pairs) . . . 3
10.1.3 Binary classification with error-correcting outputs . . . 3

10.2 Support Vector Regression (SVR) **4**
10.2.1 The ε-insensitive cost function for regression . . . 4
10.2.2 Deriving the primal problem for ε-SVR . . . 5

10.3 ν-SVR **8**
10.3.1 Motivation . . . 8
10.3.2 Deriving the primal problem for ν-SVR . . . 8
10.3.3 Deriving the Lagrangian for the primal problem of ν-SVR . . . 9
10.3.3.1 Identifying the Lagrange multipliers . . . 9
10.3.3.2 Re-arranging the constraints . . . 9
10.3.4 Optimizing ν-SVR . . . 10

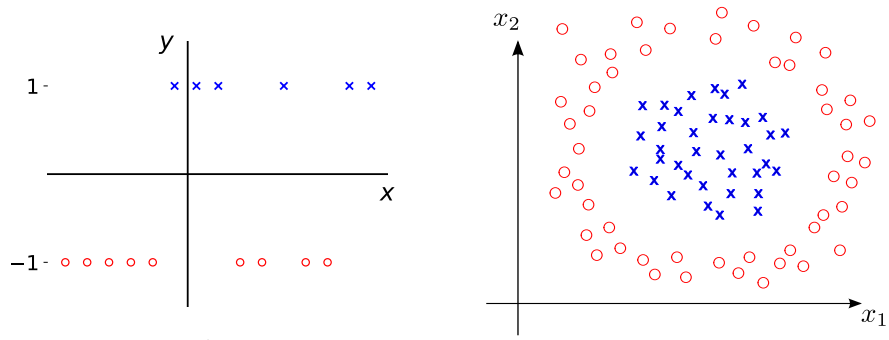
10.4 References **10**

10.1 Multi-class SVM

The binary classification setting:

Classification problems have labels with only finitely many possible values.

Example: Binary classification, $y_T \in \{-1, 1\}$:



(a) 1D input (b) 2D input

Figure 10.1.1: binary classification

A set of $K > 2$ classes: $\{c_1, c_2, \ldots, c_K\}$

Remains an open issue for SVMs.

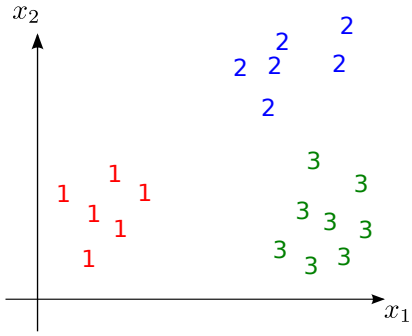


Figure 10.1.2: multi-class classification

10.1.1 One-vs-rest (one-vs-all)

for each class c_k in $\{c_1, c_2, \ldots, c_K\}$ do

train a binary classifier with c_k as its **positive** class

all remaining $K-1$ classes are treated collectively as the **negative** class.

end

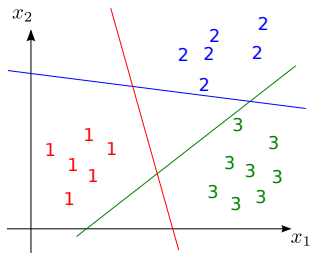


Figure 10.1.3: K one-vs-all classifiers

Q 10.1.1: *Any potential problems with training K one-vs-all classifiers?*

- ambiguous regions
- needs heuristic to disambiguate (e.g. $\operatorname{argmax}_k y_k(\underline{\mathbf{x}})$) but any heuristic needs to deal with the different scales of each $y_k(\underline{\mathbf{x}})$
- each classifier faces an imbalanced training set. In the case of SVMs, this leads to asymmetric margins. But "class weighted" SVMs exist.

Actually: one-vs-rest classification for SVMs is good enough for many problems.

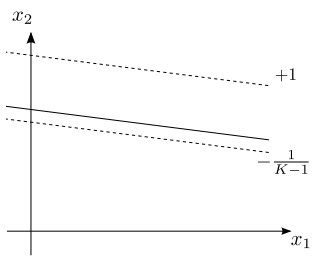


Figure 10.1.4: Asymmetric margins due to class imbalance

Class	l_1	l_2	l_3	l_4	l_5	l_6
c_1	1	-1	1	-1	-1	1
c_2	1	1	-1	-1	1	-1
c_3	-1	1	-1	1	-1	1
c_4	-1	-1	1	-1	1	1

Table 1: Example for error correcting codes using 6-bits for 4 classes

10.1.2 One-vs-one (all pairs)

binary classifiers = $\frac{K(K-1)}{2}$

:-(long test times. Testing speed can be improved by organizing the pair classifiers into a DAG[1] to reduce to a subset of $K-1$ only.

:-) Training is fast as each only requires a very small subset of the data.

:-(We still have ambiguous regions:

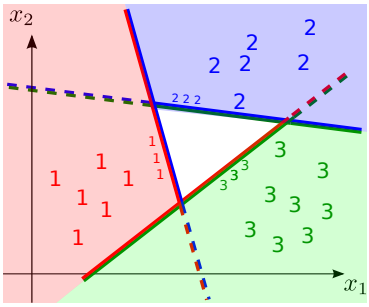


Figure 10.1.5: One-vs-one classification

10.1.3 Binary classification with error-correcting outputs

A rough outline of this ensemble method:

1. Express each class using an L-bit binary vector (e.g. $\{+1, -1\}$). Choose $L > K$.
2. Concatenate all vectors into a $K \times L$ matrix.
3. Train a binary classifier to target the value of the i-th column in this matrix.
   - If class k is associated with a "$+1$" at the i-th column. All data from this class will be used as **positive** examples when training the i-th classifier.
   - If class k is associated with a "-1" at the i-th column. All data from this class will be used as **negative** examples when training the i-th classifier.

Classifier $y_2(\underline{\mathbf{x}})$ will use all points from c_2, c_3 as positive examples and c_1, c_4 as negative examples.

By looking at the predictions of all classifiers. One would match the 6-bit prediction pattern with those of the 4 classes to determine which class was detected.

[1] Directed Acyclic Graph, a tree would qualify.

10.2 Support Vector Regression (SVR)

The regression setting:

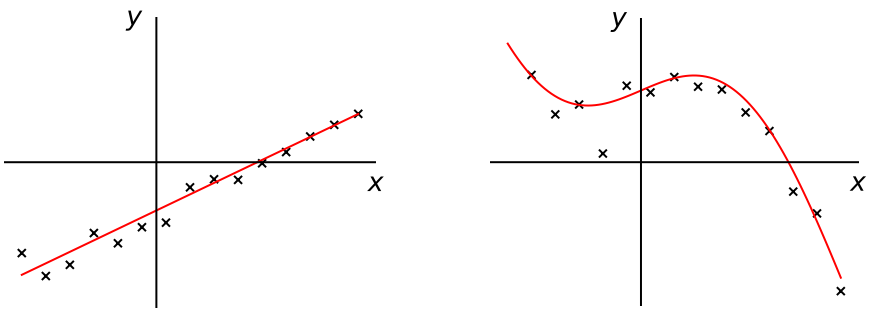
Data:

$\{(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)})\}_{\alpha=1}^{p}$ with $\underline{\mathbf{x}} \in \mathbb{R}^N$ and $y_T \in \mathbb{R}$

Model: linear neuron: $y(\underline{\mathbf{x}}; \underline{\mathbf{w}}, b) = \underline{\mathbf{w}}^\top \underline{\mathbf{x}} + b$

Objective:

- ERM: $\min_{\underline{\mathbf{w}},b} E^T$
- SRM: $\min_{\underline{\mathbf{w}},b} \frac{1}{2}\|\underline{\mathbf{w}}\|^2 + CR_{\text{emp}}(\underline{\mathbf{w}}, b)$



(a) linear regression (b) non-linear regression

Figure 10.2.1: scalar regression with 1D input

10.2.1 The ε-insensitive cost function for regression

$$e(y(\underline{\mathbf{x}}; \underline{\mathbf{w}}, b), y_T; \varepsilon) = e(\underbrace{y(\underline{\mathbf{x}})}_{\text{for brevity}}, y_T; \varepsilon) = \max(\,0\,, \left|y(\underline{\mathbf{x}}) - y_T\right| - \varepsilon\,) \quad (10.2.1)$$

- The cost function is parameterized by ε that is chosen a priori.

$\rightsquigarrow$ no error for $y(\underline{\mathbf{x}}) \in [y_T - \varepsilon, y_T + \varepsilon]$ otherwise,

$\rightsquigarrow$ linear error

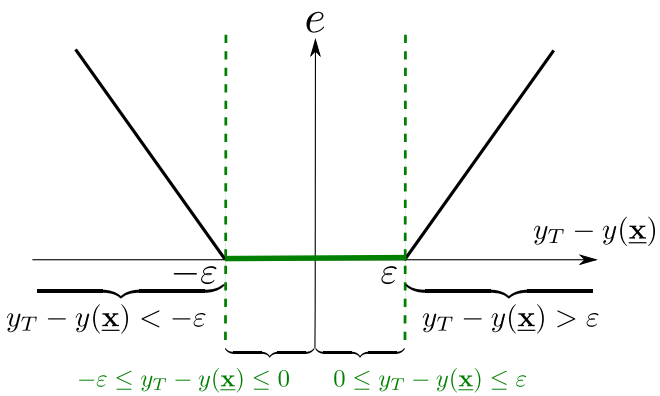


Figure 10.2.2: The cost function

The SRM objective becomes:

$$\min_{\underline{\mathbf{w}},b} \frac{1}{2}\|\underline{\mathbf{w}}\|^2 + CR^{\varepsilon}_{\text{emp}}(\underline{\mathbf{w}}, b) \quad (10.2.2)$$

using

$$R^{\varepsilon}_{\text{emp}}(\underline{\mathbf{w}}, b) = \frac{1}{p}\sum_{\alpha=1}^{p} e(y(\underline{\mathbf{x}}), y_T; \varepsilon) \tag{10.2.3}$$

$$= \frac{1}{p}\sum_{\alpha=1}^{p} \max(0, \left|y(\underline{\mathbf{x}}) - y_T\right| - \varepsilon) \tag{10.2.4}$$

10.2.2 Deriving the primal problem for ε-SVR

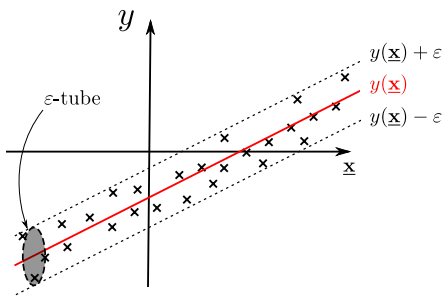


Figure 10.2.3: Linear model (no points outside the margin)

(i) Simplify assumptions (will be relaxed later)[2]

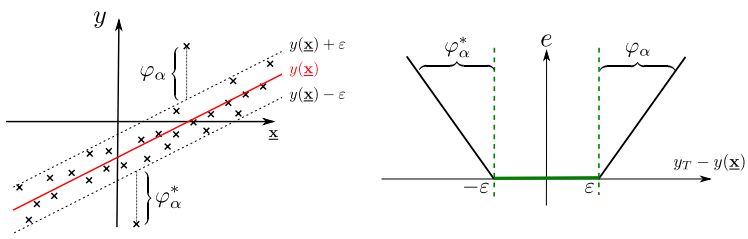
1. perfect regression solution, i.e. zero error ε-insensitive cost,
2. the constrained regression problem

$$\begin{array}{ll} \min_{\underline{\mathbf{w}},b} & \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 \\ \text{subject to} & y(\underline{\mathbf{x}}^{(\alpha)}) - y_T^{(\alpha)} \leq \varepsilon \\ & y_T^{(\alpha)} - y(\underline{\mathbf{x}}^{(\alpha)}) \leq \varepsilon \quad \text{for } \alpha = 1, \ldots, p \end{array} \tag{10.2.5}$$

Substituting $y(\underline{\mathbf{x}}^{(\alpha)}) = \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b$ yields:

$$\begin{array}{ll} \min_{\underline{\mathbf{w}},b} & \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 \\ \text{subject to} & \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b - y_T^{(\alpha)} \leq \varepsilon \\ & y_T^{(\alpha)} - \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - b \leq \varepsilon \quad \text{for } \alpha = 1, \ldots, p \end{array} \tag{10.2.6}$$

[2]This is based on my understanding of Ch. 9 from (Scholkopf and Smola, 2001).



(a) (b)

Figure 10.2.4: Relaxed assumptions

(ii) Relax constraints by allowing for errors

Q 10.2.1: *Why would we ever want to allow for errors?*

- in order to reduce overfitting to noise or outliers in the training data.

1. possibly more robust to noise and outliers,
2. constrained regression problem that allows for errors (with slack variables φ_α)

$$\begin{array}{ll} \min_{\underline{\mathbf{w}},b,\{\varphi_\alpha\},\{\varphi^*_\alpha\}} & \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 + C\frac{1}{p}\sum_{\alpha=1}^{p}(\varphi_\alpha + \varphi^*_\alpha) \\ \text{subject to} & \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b - y_T^{(\alpha)} \leq \varepsilon + \varphi_\alpha \\ & y_T^{(\alpha)} - \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - b \leq \varepsilon + \varphi^*_\alpha \\ & \varphi_\alpha, \varphi^*_\alpha \geq 0 \qquad \text{for } \alpha = 1, \ldots, p \end{array} \tag{10.2.7}$$

with $C > 0$ (C penalizes model error)

Q 10.2.2: *How do we prevent solutions with arbitrarily large $\varphi_\alpha, \varphi^*_\alpha$ that prevent any violation of the constraints?*

- The second term in the minimization objective that is parameterized by C takes care of this, as it penalizes model error.

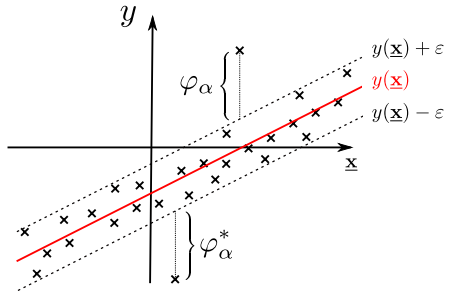
Eq.(10.2.7) gives us the *primal problem* for ε-SVR.

Q 10.2.3: *Should we divide the sum by 2 since we're adding $(\varphi_\alpha + \varphi^*_\alpha)$ inside the sum?*

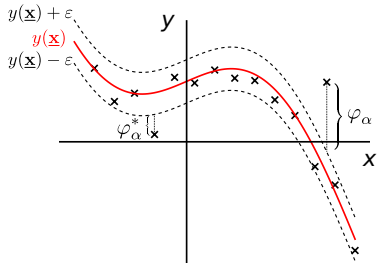
- No, this is not necessary. φ_α and φ^*_α describe error on opposite sides of the line we are fitting. A point cannot fall below **and** above a line at the same time. One of the slack variables for any point will always be zero.

At this point we can define the empirical risk term $R^\varepsilon_{\text{emp}}$ in Eq. (10.2.2) in terms of the slack variables:

$$\min_{\underline{\mathbf{w}},b,\{\varphi_\alpha\},\{\varphi^*_\alpha\}} \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 + C \overbrace{\frac{1}{p}\sum_{\alpha=1}^{p}(\varphi_\alpha + \varphi^*_\alpha)}^{=:R^\varepsilon_{\text{emp}}(\underline{\mathbf{w}},b)} \tag{10.2.8}$$



(a) linear regression



(b) non-linear regression

Figure 10.2.5: SVR for linear and non-linear regression

$$\underline{\mathbf{w}}^* = \sum_{\alpha=1}^{p} (\lambda_\alpha - \lambda^*_\alpha)\underline{\mathbf{x}}^{(\alpha)} \overset{\substack{\text{either} \\ \lambda_\alpha \text{ or } \lambda^*_\alpha \neq 0 \\ \text{for SVs}}}{=} \sum_{\beta \in \text{SV}} (\lambda_\beta - \lambda^*_\beta)\underline{\mathbf{x}}^{(\beta)} \tag{10.2.9}$$

$$y(\underline{\mathbf{x}}) = b^* + \sum_{\beta \in \text{SV}} (\lambda_\beta - \lambda^*_\beta) \underbrace{\left(\underline{\mathbf{x}}^{(\beta)}\right)^\top \underline{\mathbf{x}}}_{\text{kernels!}} \tag{10.2.10}$$

10.3 ν-SVR

10.3.1 Motivation

Choosing ε for ε-SVR can be difficult as it depends on the noise in the data and this noise is unknown. ε-SVR requires treating ε as a hyperparameter. We therefore resort to methods such as cross-validation for choosing one value for ε over another.

ν-SVR extends ε-SVR, by allowing the width of the "ε-tube" to adapt to the data. This is done by adding ε to our set of *primal variables*. We can then optimize w.r.t. ε as we do with $\underline{\mathbf{w}}$, b and the set of slack variables $\{\varphi_\alpha\}, \{\varphi_\alpha^*\}$.

The SRM objective for ν-SVR becomes:

$$\min_{\underline{\mathbf{w}},b,\{\varphi_\alpha\},\{\varphi_\alpha^*\},\varepsilon} \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 + C\left[\nu\varepsilon + \frac{1}{p}\sum_{\alpha=1}^{p}(\varphi_\alpha + \varphi_\alpha^*)\right] \tag{10.3.1}$$

10.3.2 Deriving the primal problem for ν-SVR

$$\begin{array}{lll} \min_{\underline{\mathbf{w}},b,\{\varphi_\alpha\},\{\varphi_\alpha^*\},\varepsilon} & \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 + C\left[\nu\varepsilon + \frac{1}{p}\sum_{\alpha=1}^{p}(\varphi_\alpha + \varphi_\alpha^*)\right] & \\ \text{subject to} & \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b - y_T^{(\alpha)} \leq \varepsilon + \varphi_\alpha & \\ & y_T^{(\alpha)} - \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - b \leq \varepsilon + \varphi_\alpha^* & \\ & \varphi_\alpha, \varphi_\alpha^* \geq 0 & \text{for } \alpha = 1, \ldots, p \\ & \varepsilon \geq 0 & \end{array} \tag{10.3.2}$$

with $C > 0$ (C penalizes model error), $\nu \geq 0$

ν-SVR extends ε-SVR with: $1\times$ new variable, $1\times$ new hyper-parameter, $1\times$ new constraint.

Q 10.3.1: *Aren't we just exchanging one hyperparameter for another? How is this better?*

- Yes, but with increased resolution of how we balance model error and model complexity. ν acts as a

- upper bound on the fraction of errors: $\nu \geq \frac{1}{p}\sum_{\alpha=1}^{p}(\varphi_\alpha + \varphi_\alpha^*)$ and as a
- lower bound on the fraction of SVs: $\nu \leq \frac{\#SV}{p}$[3].

[3] If interested, see details in Ch. 9.3 from (Scholkopf and Smola, 2001) and the paper (Schölkopf et al., 2000)

10.3.3 Deriving the Lagrangian for the primal problem of ν-SVR

10.3.3.1 Identifying the Lagrange multipliers

Q 10.3.2: *Where are multipliers needed?*

The primal problem

$$\begin{array}{lll}
\min_{\underline{\mathbf{w}},b,\{\varphi_\alpha\},\{\varphi^*_\alpha\},\varepsilon} & \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 + C\big[\nu\varepsilon + \frac{1}{p}\sum_\alpha^p(\varphi_\alpha + \varphi^*_\alpha)\big] & \\
\text{subject to} & \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b - y_T^{(\alpha)} \leq \varepsilon + \varphi_\alpha & \rightsquigarrow \{\lambda_\alpha\} \\
& y_T^{(\alpha)} - \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - b \leq \varepsilon + \varphi^*_\alpha & \rightsquigarrow \{\lambda^*_\alpha\} \\
& \varphi_\alpha \geq 0 & \rightsquigarrow \{\eta_\alpha\} \\
& \varphi^*_\alpha \geq 0 \qquad \text{for } \alpha = 1,\ldots,p & \rightsquigarrow \{\eta^*_\alpha\} \\
& \varepsilon \geq 0 & \rightsquigarrow \delta
\end{array} \tag{10.3.3}$$

with $C > 0$ and $\nu \geq 0$.

Next: re-arrange the constraints to be in the form $f_\alpha \leq 0$ to form the Lagrangian of the primal problem.

10.3.3.2 Re-arranging the constraints

Next: re-arrange the constraints to be in the form $f_\alpha \leq 0$ to form the Lagrangian of the primal problem:

The primal problem with constraints in the form of $f_\alpha \leq 0$

$$\begin{array}{llll}
\min_{\underline{\mathbf{w}},b,\{\varphi_\alpha\},\{\varphi^*_\alpha\},\varepsilon} & & \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 + C\big[\nu\varepsilon + \frac{1}{p}\sum_\alpha^p(\varphi_\alpha + \varphi^*_\alpha)\big] & \\
\text{subject to} & - & \{\varphi_\alpha + \varepsilon + y_T^{(\alpha)} - \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - b\} \leq 0 & \rightsquigarrow \{\lambda_\alpha\} \\
& - & \{\varphi^*_\alpha + \varepsilon - y_T^{(\alpha)} + \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\} \leq 0 & \rightsquigarrow \{\lambda^*_\alpha\} \\
& - & \varphi_\alpha \leq 0 & \rightsquigarrow \{\eta_\alpha\} \\
& - & \varphi^*_\alpha \leq 0 \qquad \text{for } \alpha = 1,\ldots,p & \rightsquigarrow \{\eta^*_\alpha\} \\
& - & \varepsilon \leq 0 & \rightsquigarrow \delta
\end{array} \tag{10.3.4}$$

with $C > 0$ and $\nu \geq 0$.

$$\begin{aligned}
L(\underbrace{\underline{\mathbf{w}}, b, \{\varphi_\alpha\}, \{\varphi^*_\alpha\}, \varepsilon}_{\text{primal variables}}, \underbrace{\{\lambda_\alpha\}, \{\lambda^*_\alpha\}, \{\eta_\alpha\}, \{\eta^*_\alpha\}, \delta}_{\text{dual variables (Lagrange multipliers)}}) \\
= \tfrac{1}{2}\|\underline{\mathbf{w}}\|_2^2 + C\Big[\nu\varepsilon + \tfrac{1}{p}\sum_{\alpha=1}^p(\varphi_\alpha + \varphi^*_\alpha)\Big] \\
- \sum_{\alpha=1}^p \lambda_\alpha\{\varphi_\alpha + \varepsilon + y_T^{(\alpha)} - \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - b\} \\
- \sum_{\alpha=1}^p \lambda^*_\alpha\{\varphi^*_\alpha + \varepsilon - y_T^{(\alpha)} + \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\} \\
- \sum_{\alpha=1}^p \eta_\alpha\varphi_\alpha - \sum_{\alpha=1}^p \eta^*_\alpha\varphi^*_\alpha - \delta\varepsilon
\end{aligned} \tag{10.3.5}$$

with $\{\lambda_\alpha\}, \{\lambda^*_\alpha\}, \{\eta_\alpha\}, \{\eta^*_\alpha\}, \delta \geq 0$

10.3.4 Optimizing ν-SVR

1. We start with optimizing the Lagrangian of the *primal problem*. As we did with SVMs and C-SVMs, this is achieved by
   (a) calculating the derivatives of L w.r.t. *primal variables* $\underline{\mathbf{w}}, b, \{\varphi_\alpha\}, \{\varphi^*_\alpha\}, \varepsilon$
   (b) setting the partial derivatives to zero. (**see homework**)
2. Substituting the solutions to the *primal problem* back into the Lagrangian yields the Lagrangian of the *dual problem*, which can be expressed in terms of the *dual variables* only:

$$\max_{\{\lambda_\alpha\},\{\lambda^*_\alpha\}} -\tfrac{1}{2}\sum_{\alpha,\beta=1}^{p}(\lambda^*_\alpha-\lambda_\alpha)(\lambda^*_\beta-\lambda_\beta)(\underline{\mathbf{x}}^{(\alpha)})^\top\underline{\mathbf{x}}^{(\beta)}+\sum_{\alpha=1}^{p}(\lambda^*_\alpha-\lambda_\alpha)y_T^{(\alpha)}$$

s.t. $\forall\alpha\in\{1,\dots,p\}$:

$$0\le\lambda_\alpha\le\tfrac{C}{p}\,,\quad 0\le\lambda^*_\alpha\le\tfrac{C}{p}\,,\quad \sum_{\alpha=1}^{p}(\lambda_\alpha-\lambda^*_\alpha)=0\,,\quad \sum_{\alpha=1}^{p}(\lambda_\alpha+\lambda^*_\alpha)\le\nu C\,.$$

10.4 References

References

Bernhard Scholkopf and Alexander J Smola. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2001.

Bernhard Schölkopf, Alex J Smola, Robert C Williamson, and Peter L Bartlett. New support vector algorithms. *Neural computation*, 12(5):1207–1245, 2000. URL https://alex.smola.org/papers/2000/SchSmoWilBar00.pdf.